

Roo2:2026

Ejercicios Antlr lab

Pool (Pseudo Object Oriented Language)

Pool es un lenguaje imperativo y orientado a objetos muy simplificado,

-pensado para expresar de forma compacta comportamiento ('behaviour'): creación de objetos, acceso/asignación de campos, llamadas a métodos y expresiones aritmético-lógicas.

-

Funciones ('def')

-pueden tener o no parámetros.

-devuelven el resultado de evaluar la última expresión.

-No se distinguen variables locales de variables definidas en la clase a la cual se asocia el comportamiento. Una etapa de análisis semántico se encarga de hacer el binding correspondiente.

Sentencias ('stat')

- Asignación simple a variable: `ID = expr ;`

- Asignación a campo de un objeto: `ID.ID = expr ;`

- Expresión usada como sentencia (por ejemplo, una llamada a método): `expr ;`

-Flujo de control (condicional): se utiliza la expresion ternaria `expr ? true_exp : false_exp;`

Ej1: `a = a ? siempreEvalua() : nuncaEvalua();`

Ej2: `a < a ? nuncaEvalua() : siempreEvalua();`

Expresiones ('expr')

- Átomos: identificadores, enteros, booleanos, `this`, y llamadas a función.

- Orientación a objetos: creación de instancias (`new ID(...)`), acceso a campos (`ID.ID`), invocación de métodos (`ID.func(...)`), y sus equivalentes con `this` (`this.ID`, `this.func(...)`).

- Operadores aritméticos: potencia (`^`, asociativa a la derecha), multiplicación/división, suma/resta, y menos unario.

- Operadores lógicos: `not`, `and`, `or`.

- Operadores relacionales: `<`, `>`, `<=`, `>=`, `==`, `!=`.

- Operador ternario: `cond ? expr : expr`.

- Paréntesis para agrupar expresiones.

- La precedencia está definida implícitamente por el orden de las reglas alternativas (de mayor a menor precedencia): paréntesis/átomos → unarios → potencia → mul/div → suma/resta → comparación → and → or → ternario.

Léxico

- Palabras clave: `and`, `or`, `not`, `true`, `false`, `this`, `new`.
- Identificadores (`ID`) con la convención típica (letra o `\_` inicial, seguido de letras, dígitos o `\_`).
- Enteros (`INT`) sin soporte explícito para decimales.
- Espacios en blanco se ignoran (`skip`).
- No hay tokens para comentarios ni para cadenas de texto (strings), lo que indica que el lenguaje es minimalista, enfocado solo en lógica de control y manipulación de objetos/campos.

Ejercicios

Con cada uno de los siguientes ejemplos,

1. Considere que en el lenguaje dado por la cátedra el llamado a una función o método retorna el valor generado por la evaluación de la última sentencia (***stat***, en la gramática)
2. Determine si hay un code smell y cual es.
3. Evalúe el código en antlr lab usando las especificaciones del parser
4. Estudie el árbol generado por antlr lab
5. Escriba un pseudocódigo que permita detectar el code smell en el árbol

Nota: El contenido del archivo "pool.g4" debe ser copiado y pegado en la pestaña del parser

1

```
f(x,y) {  
    a = 4  
    a = 3 + y;  
    x * x + y * x;  
}
```

2

```
f(x, y, z) {  
    a = 3 + y;  
    x * x + y * x;  
}
```

3

```
f(x,y) {  
    a = 4;  
    x + a;  
}
```

4

```
f(x) {  
    a = 4;  
    a = 5;  
}
```

5

```
f(x) {  
    a = x ? 3 : 3;  
}
```

6

```
f(y) {  
    a = g.x() + g.x() + g.x();  
}
```

7

```
f(y) {  
    a = not not y;  
}
```

8

```
f(y, z) {  
    z + 12;  
}
```

9

```
elegirSueldo(empleado) {  
    clase = empleado.class;  
    clase.equals(pasante) ? empleado.setSueldo(20000);  
    clase.equals(planta) ? empleado.setSueldo(50000);  
}
```

10

```
agregarOnceNumeros(lista) {  
    lista.agregar(1);  
    lista.agregar(2);  
    lista.agregar(3);  
    lista.agregar(4);  
    lista.agregar(5);  
    lista.agregar(6);  
    lista.agregar(7);  
    lista.agregar(8);  
    lista.agregar(9);  
    lista.agregar(10);  
    lista.agregar(11);  
}
```

11

```
numeroTelefonoCompleto(telefono, numero) {  
    numero = telefono.codigoArea + telefono.prefijo + telefono.numero;  
}
```

12

```
f(x,y) {  
    x or not x ? y + 1;  
    x and x ? y - 1;  
    x ? x ? y - 1;  
}
```

13

```
f(x) {  
  x = x;  
}
```

14

```
f(x,y) {  
  x ? y - 1;  
  not x ? y - 2;  
}
```

15

```
f(a,b,c,d,e,f,g,h,i,j,k) {  
  a+b+c+d+e+f+g+h+i+j+k;  
}
```

16

```
someOperation(x,y,z) {  
  other.someOperation(x,y,z);  
}
```